Practical 1: Array Operations: Traversal, Rotation, andFrequency Analysis

Problem 1

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h, print the final display order.

Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

```
#include<iostream>
#include<string>
using namespace std;s
int main()
{
   int x=0;//Stores how many times the array should be rotated.
   int n=0;//Temporarily stores the first element during rotation.

  int ar[5]={1,2,3,4,5};

   for(int i=0;i<5;i++)
   {
      cout<<ar[i]<<" ";
   }
   cout<<endl;

   cout<<"Enter the number of items: ";
   cin>>x;

      for(int i=0;i<x;i++)
```

```
    {
      n=ar[0];
      for(int j=0;j<4;j++)
      {
        ar[j]=ar[j+1];
      }
      ar[4]=n;
    }
    cout<<endl;
    for(int i=0;i<5;i++)
    {
      cout<<ar[i]<<" ";
    }
  return 0;
}
```

Output screenshot:

```
1 2 3 4 5
Enter the number of items: 12

The arrangement of the bread in the shelves will be:
3 4 5 1 2
```

Description:

In this practical , we keep item in a shelf and every hour the back most item comes to front . So in x hours of time which item will be at front and what will be its arrangement.

Problem 2

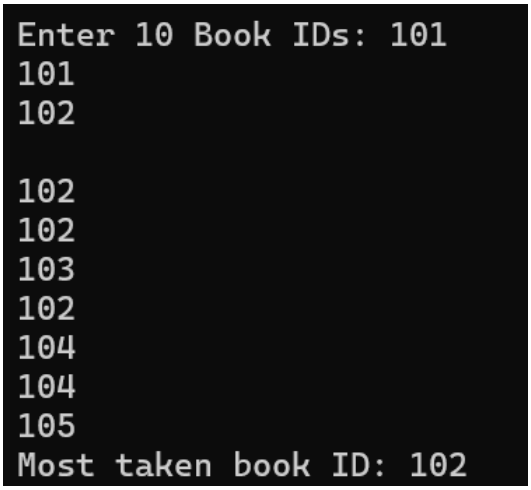
A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs.

Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

```
#include <iostream>
using namespace std;
int main()
{
   int ar[5] = {101, 102, 103, 104, 105};
   int entry[10];
   int count[5] = {0};
   cout << "Enter 10 Book IDs: ";
   for(int i=0;i<10;i++)
   {
      cin >> entry[i];
   }
   for(int i=0;i<10;i++)
   {
      for(int j=0;j<5;j++)
      {
      if(entry[i]==ar[j])
      {
```

```
        count[j]++;//if any book repeats it will be stored
            break;
        }
        }
    }

    //counting which book taken the most
    int max=count[0];
    int book=ar[0];

    for (int i=1;i<5;i++)
    {
        if (count[i]>max)
        {
            max=count[i];
            book=ar[i];
        }
    }

    cout << "Most taken book ID: " << book;

    return 0;
}
```

Output Screenshot:



Description:

In this practical we need to find the most books taken and count of the book issued form library.

Problem 3

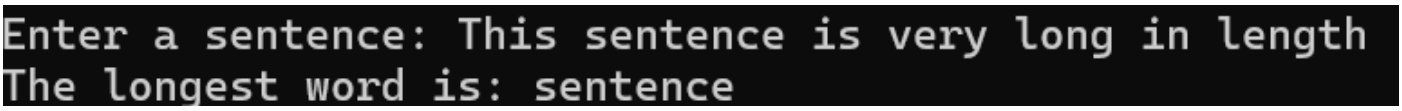
A school newspaper editor wants to highlight the most impressive word from a submitted article on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has.

Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

```
#include<iostream>
#include<string>
#include<sstream>
using namespace std;
int main()
{
   string sentence;
   cout<<"Enter a sentence: ";
   getline(cin,sentence);
   stringstream ss(sentence);     // Convert the sentence into a stream of words
   string word;                // Stores one word at a time
   string largest = "";
   while (ss >> word)           // Read one word from the sentence each time, when null exit the while loop
   {
      if (word.length() > largest.length())   // Check if current word is longer
      {
         largest = word;       //save the current word as the longest
      }
```

```
    }
    cout << "The longest word is: " << largest;
    return 0;
}
```

Output screenshot:



Description:

In this practical we need to find the longest word from the headline of the newspaper .

Questions:

1. The display order repeats itself after a certain number of hours regardless of how large h gets. Without trying any specific example, explain why this repetition is inevitable and what determines when it first repeats.

The display order repeats because there are only **n possible positions** for the leftmost item. Each hour performs a **left rotation by one position**. After n rotations, every item returns to its original position.

For the given program, n = 5, so:

- h = 0 → 1 2 3 4 5
- h = 5 → 1 2 3 4 5
- h = 10 → 1 2 3 4 5

Therefore, the first repetition occurs after **n hours**. For a very large h, such as 10 million, we do not need to perform all rotations. We can calculate:

effective rotations = h % n

Then perform only h % n rotations. This reduces the number of operations significantly.

2. If the same book ID appears three times in the log, should it appear once or

three times in your output? Does your solution handle this correctly, and how would you verify it?

If the same book ID appears three times in the borrowing log, it should be considered **one book ID that has been borrowed three times**, not printed three times. The output should contain the book ID once, with its occurrence count used to determine whether it was borrowed more than once.

The given solution correctly counts occurrences using the count[] array. However, it **does not actually print all books borrowed more than once**; instead, it prints only the **most frequently borrowed book**.

To verify the solution, we can enter a log such as:

101 102 101 103 101 104 102 105 102 102

The counts should be:

- 101 → 3
- 102 → 4
- 103 → 1
- 104 → 1
- 105 → 1

The current program should output 102 because it has the highest count. If the requirement is to print **all books borrowed more than once**, we should scan count[] and print every book where count[i] > 1.

The current solution scans the 10 entries and compares each with the 5 known book IDs, so its complexity is **O(n × m)**, where n is the number of borrowing records and m is the number of book IDs. For 100,000 records, it can still work for a small number of books, but a hash map/unordered map can count IDs in approximately **O(n)** time with one pass.

3. The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?

The current solution uses stringstream to separate the sentence into words and compares their lengths. If punctuation such as ,, ., !, or ? should not count toward the word length, punctuation should be removed **after extracting each word and before comparing its length**.

For example:

"Hello, amazing!"

should be treated as:

- Hello → length 5
- amazing → length 7

Instead of:

- Hello, → length 6
- amazing! → length 8

We can handle this by removing punctuation characters from each word using functions such as ispunct() before calculating its length. The same step should also be applied when storing the longest word, so the output contains the cleaned word.

Conclusion:

In these three problems, basic array and string processing techniques are used to solve practical data-handling tasks. The first problem demonstrates that repeated rotations are **cyclic**, so large values of h can be reduced using h % n. The second problem demonstrates **frequency counting** of book IDs and shows that a single-pass approach using a frequency table or hash map is more efficient for large data. The third problem demonstrates **string processing** to find the longest word, with punctuation removable before calculating word length. Overall, these problems illustrate how choosing an appropriate algorithm can improve both **correctness and efficiency**, especially when the input size becomes large.